ChatGPT

# Fine-Tuning Large Language Models: Mechanisms and Adaptation Strategies

## Executive Summary

Large pretrained transformers serve as the backbone of modern NLP. Adapting these **large language models (LLMs)** to new tasks or specialized datasets requires careful design of fine-tuning methods. We survey core LLM mechanisms and a spectrum of fine-tuning strategies – from full parameter updates to parameter-efficient techniques (adapters, LoRA, prefix/prompt tuning, instruction tuning, etc.) – and outline how architecture, loss/optimization, and representation interplay during adaptation. We also review dataset-specific adaptation (data augmentation, curriculum, domain- and task-adaptive pretraining) and metrics for evaluation. We summarize practical protocols (learning rates, batch sizes, checkpoints, reproducibility) and common pitfalls (overfitting, forgetting, instability). As part of this analysis, we propose **novel hybrid methods** (e.g. *conditional adapters* that modulate adapter modules by task embedding, and *self-distilling fine-tuning* that uses the original model as a teacher to stabilize learning) with expected benefits and risks (see diagrams below). A comparative table contrasts methods by parameters updated, compute cost, and use-cases. Throughout, we cite seminal sources (e.g. transformers [33], PEFT studies [10] [13][15][17][19][48]) and recent advances. Finally, we list open questions (e.g. multi-task generalization, efficient transfer theory) and benchmarks (GLUE/SuperGLUE, MMLU, etc.) to assess novel adaptation approaches.

## Transformer Architecture and Transfer Foundations

Modern LLMs are overwhelmingly based on the **Transformer** architecture [33]. A typical Transformer stacks multi-head self-attention and feedforward sublayers with residual connections and layer norms. *Encoder-only* models (e.g. BERT, RoBERTa) perform bidirectional encoding, while *decoder-only* models (e.g. GPT) generate text autoregressively; encoder–decoder models (e.g. T5, BART) combine both. These architectures are highly over-parameterized (billions to trillions of weights) to capture wide linguistic knowledge【9†L142-L150】【25†L1339-L1345】.

In a standard workflow, a Transformer is first **pretrained** on large text corpora (unsupervised language modeling or denoising). It learns general linguistic representations (syntax, semantics) in its layers. Then for a downstream task, we **fine-tune** the model with a task-specific objective. Early layers tend to retain general features, while deeper layers adapt to task-specific concepts【5†L479-L485】. Indeed, recent probing shows that fine-tuning mainly alters higher layers: the similarity between pretrained and fine-tuned activations drops sharply in the top layers, whereas bottom layers remain mostly unchanged【5†L479-L485】. This suggests a *hierarchical representation*: shallow layers encode universal patterns, deep layers specialize. The concept of **intrinsic dimensionality** further quantifies this: Aghajanyan et al. (2021) find that optimizing only a very low-dimensional subspace (e.g. ~200 directions) suffices to achieve ~90% of full fine-tune performance【43†L59-L67】. Larger models tend to have even lower intrinsic dimension after pretraining, which may explain why they adapt with fewer updates【43†L59-L67】.

**Loss functions** for LLM fine-tuning are usually standard: cross-entropy on the next-token or label prediction. Common practice adds **label smoothing** (e.g. $\epsilon$=0.1) to improve generalization【35†L471-L479】. Task-specific heads (classification/regression layers) are often added and trained from scratch. Regularization (dropout, weight decay) and learning-rate schedules (warmup, decay) are also used. **Optimization** is typically done with AdamW (Adam with decoupled weight decay) and small learning rates (often 1e-5–5e-5) to gently nudge the pretrained weights. A small number of epochs (2–4 on moderate-sized tasks) is common to avoid overfitting. Gradients are clipped to prevent large updates.

From a transfer-learning perspective, fine-tuning can be viewed as *constrained optimization* starting from a pretrained optimum. In fact, parameter-efficient methods interpret fine-tuning as optimizing in a low-dimensional *delta* subspace of parameters【10†L49-L57】. Ding et al. (2023) formalize many PEFT methods under a "delta-tuning" framework: only a small fraction of parameters are updated while the rest remain frozen【10†L19-L27】【10†L49-L57】. This constraint helps preserve pre-trained knowledge. Intuitively, the pretrained LLM provides a strong inductive bias, and fine-tuning adjusts it to the new data. If the new task is similar to pretraining domains, minimal changes are needed, but if the domain shifts, more adaptation is required (see **domain adaptation** below). Over-large updates can cause *catastrophic forgetting*, where the model loses generality; small learning rates and techniques like mixout or regularization help mitigate this.

## Fine-Tuning Methods

- **Full fine-tuning.** The simplest approach is to update *all* model parameters on the new task (often adding a small task-specific head). This has the highest flexibility and generally the best possible performance when ample data is available. However, it is *compute- and memory-intensive* (requiring gradients/updates for 100% of weights) and needs storing a full model copy per task. BERT (2018) and early GPTs fine-tuned in this way for each task. Fine-tuning large models (e.g. GPT-3 175B) can take days on powerful clusters.

- **Adapter modules.** Adapters insert small trainable layers into each transformer block while *freezing* the original weights【13†L50-L59】. A typical adapter is a bottleneck (down-projection, nonlinearity, up-projection). Only these adapters are trained per task. Houlsby et al. (2019) report adapter-tuned BERT achieves within ~0.4% of full fine-tuning performance on GLUE, while adding only ~3.6% extra parameters per task【13†L50-L59】【13†L60-L63】. This dramatically reduces storage: e.g. separate tasks share the base model and only differ by small adapter weights. Extensions include *AdapterFusion* (combining adapters from multiple tasks) and *Conditional Adapters* (modulating adapters by task signals)【52†L52-L61】. Lei et al. (2023) show a conditional-adapter variant ("CoDA") can even speed up inference by sparsely activating adapters with minimal accuracy loss【52†L52-L61】.

- **LoRA (Low-Rank Adapters).** LoRA freezes the main weights and adds trainable low-rank matrices that model the weight updates【15†L54-L63】. Concretely, a weight matrix $W$ in a transformer is updated as $W + \Delta W$, where $\Delta W= A B$ and $A,B$ are small (rank-$r$) matrices trained on the new task. LoRA greatly reduces trainable parameters. For example, fine-tuning GPT-3 (175B) traditionally updates 175,255M parameters, but LoRA only needs on the order of 37.7M ($\approx$0.02%) to match performance【10†L25-L33】. Hu et al. (2021) report LoRA reduces GPU memory by ~3× and supports on-par or better quality with no inference slowdown【15†L60-L67】. Variants include **AdaLoRA** (dynamically adjusting rank per layer) and **QLoRA**: Dettmers et al. (2023) combine 4-bit

quantization of the base model with LoRA, enabling fine-tuning of a 65B model on a single 48GB GPU without losing performance【54†L53-L61】. (QLoRA preserves the 16-bit fine-tune accuracy by careful quantization and adapter placement.)

- **Prefix/P-tuning.** These are *prompt-based* methods that prepend trainable vectors to inputs or each layer. **Prefix-tuning** (Li & Liang 2021) introduces a sequence of continuous "prefix" tokens to each layer's attention context, keeping the model frozen. Only the prefix vectors are learned. They found ~0.1% of parameters (the prefixes) suffice to match full fine-tuning performance on generation tasks【17†L59-L64】, and that prefix-tuning can even generalize better in low-data or out-of-domain cases. **Prompt-tuning** (Lester et al. 2021) is a simpler case: only the input embedding layer's new prompt tokens are trained. Prompt-tuning performance improves with model scale: on T5 models exceeding billions of parameters, learned soft prompts essentially "close the gap" with full fine-tuning【19†L58-L61】. In practice, prefix/prompt methods work best on very large decoders (GPT, T5) for tasks like generation or classification framed as fill-in.

- **Instruction tuning.** In contrast to methods that adapt to a single task, *instruction tuning* fine-tunes on a collection of tasks described in natural language. By training a model to follow instructions on many examples, it gains few-shot or zero-shot abilities on new tasks. For example, Wei et al. (2022) "instruction-tune" a 137B model on 60+ tasks (called FLAN), and it dramatically improves zero-shot accuracy on unseen tasks, even outperforming 175B GPT-3 on many benchmarks【48†L52-L60】. OpenAI's InstructGPT (2022) uses supervised fine-tuning on human-instructed data plus RLHF to align GPT-3 with user intent. Instruction tuning usually involves updating a significant portion of parameters (often full fine-tuning) on a large, diverse dataset of (input, instruction, output) triples, so it carries similar costs as full fine-tuning but yields a broadly capable model.

- **Other PEFT variants.** Many hybrids exist. **Compacter** and **MAM adapters** shrink adapter size with tensor decomposition. **BitFit** trains only the bias terms in each layer. **LoRA + Adapter** combinations have been tried (jointly training both). **Prefix + Adapter** hybrids could insert both trainable prefixes and adapter layers. Each trades off parameter count, speed, and flexibility. The recent overview by Ding et al. calls all of these "delta-tuning" methods and finds that combining methods often helps【10†L79-L88】. Generally, PEFT methods update <1–5% of parameters per task, yielding large savings in storage and often comparable accuracy to full fine-tuning if the base model is large【10†L25-L34】【10†L79-L88】.

- **Continual learning.** For sequential tasks, the goal is to avoid forgetting old tasks when fine-tuning on a new one. Techniques include **regularization** (Elastic Weight Consolidation, i.e. EWC, penalizes changes to important weights) and **replay methods** (periodically retraining on old data or synthetic pseudo-data). Adapters/LoRA naturally help here: one can keep separate adapters per task, or augment past adapters with new ones. Recent work applies LLM adaptation with continual constraints, but this area is still maturing.

- **Multi-task and meta-learning.** Fine-tuning on multiple tasks at once (multi-task learning) can improve generality but may require careful balancing (task re-weighting, shared vs private heads). Meta-learning (e.g. MAML, meta-gradient approaches) has been explored for rapid adaptation, but scaling it to LLMs is challenging. Some efforts (e.g. PETL-meta) adapt adapter parameters via meta-learning to accelerate few-shot learning. These approaches are promising but less mature for large models.

**In summary**, the landscape ranges from heavy "update everything" (full FT, often ideal with massive data) to ultra-light "learn a tiny tweak" (prefix/prompt, LoRA, etc.). Table 1 below (in **Method Comparison**) summarizes key trade-offs (parameters updated, compute, sample needs, etc.). Notably, researchers find that *any* sufficiently large model can be steered by only a few trainable parameters: prefix/prompt (~0.1%), adapters (~1–5%), LoRA (~0.01%+)【15†L60-L67】【17†L59-L64】 can achieve non-trivial performance across many tasks. This has enabled efficient sharing of a single backbone for many tasks via small "delta" files.

## Dataset and Task-Specific Adaptation

Fine-tuning must account for the *nature of the data*. Several strategies help align models to specific datasets:

- **Data augmentation:** Creating synthetic data to enlarge the training set. In vision, standard flips/crops exist; for text, common methods include *synonym replacement, back-translation, paraphrasing*, or even *model-generated* examples. Simple techniques (Wei & Zou's "EDA" [TODO]) involve word swaps/insertions. More powerful is using the LLM itself: e.g. GPT-based paraphrasing or retrieving similar sentences from corpora. Surveys categorize augmentation into (1) *simple transformations* (token or sentence edits), (2) *prompt-based generation* (instructing an LLM to create variations), (3) *retrieval-based* (find external sentences from large data), and (4) *hybrid* methods【55†L165-L173】. Augmentation can improve generalization especially in low-resource settings; for example, prompting GPT-3 to generate paraphrases or new QA pairs has boosted few-shot performance. Care is needed to filter out noise or ungrammatical samples. Overall, "data augmentation is a critical technique to improve LLM performance by expanding existing data"【55†L165-L173】.

- **Curriculum learning:** Ordering or weighting examples from easy to hard. In principle, training on simpler examples first can stabilize learning. In practice, this may involve gradually unfreezing layers (as in ULMFiT) or starting with high-confidence samples. Though promising, curriculum strategies for LLM fine-tuning are less standardized than in vision. A related idea is *incremental task complexity*: begin fine-tuning on more general tasks or domains, then refine on specific subdomains.

- **Domain adaptation:** When the target data's distribution differs from pretraining (e.g. medical text vs web text), performance can suffer. A common remedy is **Domain-Adaptive Pre-Training (DAPT)**: further pretrain the LLM on large unlabeled in-domain text before fine-tuning【38†L23-L32】. Gururangan et al. (2020) show that continuing RoBERTa pretraining on domain text (news, biomedical, reviews, etc.) yields significant gains on downstream tasks, beyond what task-tuning alone achieves【38†L23-L32】. A step beyond is **Task-Adaptive Pre-Training (TAPT)**: even using the task's unlabeled inputs for masked-language modeling, which further boosts accuracy【38†L23-L32】. When unlabeled in-domain data is limited, selecting a subset of the pretraining corpus similar to the target (e.g. via KL divergence or retrieval) can help. Features like domain tokens or vocabulary adjustments are also used for domain shifts.

- **Few-shot and example selection:** When labeled data is very scarce (few-shot), strategies include *prompting* with exemplar examples (in-context learning) or training small prompt/adapter modules. During fine-tuning, one can employ *meta-learning* to prepare the model for few-shot, or use *active learning* to pick the most informative samples to label. Another tactic is to augment few-shot data by

generating paraphrases or related examples using the model itself. Ensuring that examples cover the task's "challenges" (diversity of inputs, label balance) is crucial.

- **Label noise handling:** Real datasets often have annotation errors. Robust training methods like label smoothing, confidence-based example re-weighting, or noise-robust losses (e.g. bootstrapping where the model's own predictions are mixed with labels) can mitigate this. Early stopping and larger regularization can also help prevent the model from memorizing incorrect labels. If noise is suspected, one can iteratively clean labels using the model's predictions or use a small clean validation set to adjust.

- **Evaluation metrics:** Choice depends on the task. *Automatic metrics* include accuracy/F1/precision/recall for classification, exact match or F1 for QA span extraction, BLEU/ROUGE/METEOR for generation (translation, summarization), and perplexity/log-likelihood for language modeling. Intrinsic metrics like embedding similarity (BERTScore, MoverScore) are also used. For open-ended generation (dialogue, summarization), **human evaluation** is common, judging consistency, coherence, relevance, and factuality【25†L1345-L1349】. (Human eval is costly but sometimes necessary for chatbots or creative tasks.) Benchmarks often specify the relevant metric: e.g. GLUE/SuperGLUE use accuracy/F1; summarization uses ROUGE; generation can use BLEU or human-rated quality【25†L1339-L1345】【25†L1345-L1349】.

## Experimental Protocols and Best Practices

Building a robust fine-tuning pipeline involves careful hyperparameter tuning and reproducibility measures:

- **Hyperparameters:** Typical settings (in examples like BERT) are: learning rate ~1–5×10^-5, batch size 16–32, warmup steps (e.g. 10% of steps), up to 3–5 epochs for classification, up to ~1 epoch for large generative tasks. Weight decay ~0.01, dropout ~0.1. Layerwise learning rates (smaller rates for base vs new head) and *gradual unfreezing* (start by training only top layers, then lower ones) can improve stability. Optimizers: AdamW (β=(0.9,0.999)) is standard. Gradient clipping (norm ~1) prevents gradient explosion.

- **Compute and checkpoints:** Fine-tuning budgets scale with model size. For example, tuning a 100M–1B model may take minutes–hours on a single GPU, whereas a 100B+ model could require multiple GPUs or TPU pods. When using adapters/LoRA, memory usage is much lower than full FT (LoRA can reduce memory by ~3×【15†L60-L67】, QLoRA even 4-bit quantizes weights【54†L53-L61】). Regularly save checkpoints (e.g. every epoch or every 1000 steps) so you can resume or roll back if needed. For parameter-efficient methods, store both the base model and the small adapter weights; many frameworks (Hugging Face PEFT) automate this. Use mixed precision (fp16) if supported to speed up training.

- **Reproducibility:** Set random seeds for all libraries (Torch, NumPy, etc.) to ensure repeatability. Log and publish all hyperparameters, training curves, data splits. If possible, report multiple runs (especially in low-data settings) to capture variance. Use deterministic operations if exact reproducibility is needed (though this can slow training). Containerization (Docker) or environment management ensures library versions are consistent.

- **Ablation studies:** To understand what works, ablate one component at a time. For example, compare full vs adapter vs LoRA vs prompt by keeping the same data and seed. Vary adapter size/rank, prompt length, learning rate, etc., to gauge sensitivity. Always include a baseline (often the original pretrained model's zero/few-shot performance).

- **Failure modes:** Common issues include catastrophic forgetting (performance drop on earlier tasks), instability (loss spikes), and overfitting (large gap between train and validation performance). Monitor both training and held-out task metrics. If loss diverges, reduce learning rate or regularize more. If fine-tuned model hallucinates or becomes unsafe (for chatbots), apply post-hoc filtering or RLHF if alignment is needed. In multi-domain training, ensure one domain doesn't dominate at the expense of others (use balanced sampling or loss weighting).

## Proposed Novel Adaptation Methods

Based on the survey above, we propose two example hybrid fine-tuning ideas:

```
flowchart LR
    subgraph Conditional Adapter Tuning
      T[Task/Domain Embedding] --> G[Gate Network]
      I[Input Tokens] --> E[Transformer Encoder\n(Base, frozen)]
      G -->|modulates| A[Adapter Module (per layer)]
      E --> A --> O[Output]
    end
```

**Conditional Adapter Tuning:** We insert adapter modules into each Transformer layer, but instead of static parameters, the adapter's behavior is *conditioned* on a learnable task/domain embedding. Concretely, let each adapter's weights be the output of a small neural *gating network* that takes as input a task identifier vector $z$. This allows one base model to adapt flexibly to multiple tasks: by changing $z$, the same adapter layers produce different transformations. *Rationale:* Traditional adapters use separate weights per task; a conditional version shares weight *generators* but still tailors adaptation. This can improve parameter sharing across tasks (fewer total parameters if tasks are similar) and enable smooth interpolation between domains. *Expected benefits:* better multi-task efficiency, smoother transfer when adding new tasks (just learn a new $z$ rather than new adapters). *Risks:* Complexity of training (two nets: gate and adapter) and potential underfitting if the gating net is weak. *Implementation steps:* initialize a small embedding vector per task, train its parameters jointly with the gating network to produce adapter weights. One could start with a fixed $z$ per known task, and extend by meta-learning to new tasks.

```
flowchart LR
    subgraph Self-Distilled Fine-Tuning
        X[Input Text] --> M0[Original LLM (frozen)]
        X --> M1[Fine-Tuned LLM]
        M0 -->|Soft Labels| C[Cross-Entropy Loss L_{KD}]
        M1 -->|Predictions| C
        M1 --> D[Task Head]
        D --> Y[Loss L_{task}]
```

```
        C & Y --> Loss[Total Loss = L_task + λ L_KD]
    end
```

**Self-Distillation Fine-Tuning:** During fine-tuning, in addition to the usual task loss (e.g. cross-entropy on labels), we **distill** the original pretrained model's outputs into the fine-tuned model. In practice, pass each input through both the frozen original LLM and the adapting model; compute a soft-KL or cross-entropy loss between their output distributions ("teacher" vs "student"). The final loss is a weighted sum of task loss and distillation loss. *Rationale:* This anchors the fine-tuned model to the pretrained distribution, preventing overfitting or catastrophic shifts in representation. It is akin to L2-SP or EWC but at the output level. *Expected benefits:* Maintains general knowledge (reduces forgetting) and smooths training (the teacher acts as a stabilizer). *Risks:* Might slow convergence on the new task (since the model is pulled towards old behavior); requires tuning the distillation weight λ. *Implementation:* Pre-compute or batch-process teacher logits for training data. Use typical distillation techniques (temperature scaling). Test ablations with λ=0 (no distill) to see the benefit.

```
flowchart LR
    subgraph Hybrid LoRA+Prefix
        X[Input Tokens] --> E[Transformer Base (frozen)]
        E --> P[Prefix Tokens (trainable)] --> A[Attention]
        E --> Y[LoRA Modules (trainable)] --> O[Output]
    end
```

**LoRA+Prefix Hybrid:** Combine low-rank weight updates (LoRA) with prefix embeddings. For instance, in each multi-head attention layer, prepend a small trainable prefix (as in prefix-tuning) *and* apply LoRA adapters to query/key projections. This provides two complementary fine-tuning "handles": one modifies input context (prefix), the other directly alters weights (LoRA). *Rationale:* Prefix vectors can quickly adjust model behavior, while LoRA captures deeper parameter shifts. Their combination may enable faster learning with minimal parameters. *Benefits:* Potentially faster convergence (prefix gives large initial move, LoRA fine-tunes), and robustness (one can ablate if one method overshoots). *Risks:* More hyperparameters (learning rates for prefix vs LoRA) and slightly increased training cost. *Implementation:* Initialize both modules randomly, jointly train with combined loss.

These are illustrative examples; full development would involve thorough experiments and comparisons.

## Method Comparison

| Method | Trainable Params (% of model) | Compute/ Memory | Data Efficiency | Typical Use-Cases |
|---|---|---|---|---|
| **Full Fine-Tuning** | 100% | High (updates all weights, large GPU memory) | Requires large labeled dataset; can overfit small data | When ample data and compute; best final accuracy |

| Method | Trainable Params (% of model) | Compute/ Memory | Data Efficiency | Typical Use-Cases |
|---|---|---|---|---|
| **Adapters** | ~3–5% (e.g. +3.6% per task)【13†L50-L59】 | Moderate (slight forward/backward overhead) | Good even for moderate data; robust (freeze base) | Multi-task/domain adaptation; few-shot tasks |
| **LoRA** | ~~0.01–0.1% (10,000× fewer params) 【15†L60-L67】~~ Actually GPT-3: ~0.02%【10†L25-L33】 | Low (no additional inference cost; 3× less GPU mem) 【15†L60-L67】 | Very high (works well even with limited labels) | Large model fine-tuning on single tasks; when storage/compute limited |
| **Prefix-tuning** | ~0.1% (fixed prefix vectors)【17†L59-L64】 | Very low (frozen base, small vectors added) | Moderate: needs sufficiently large base model; helps few-shot 【17†L59-L64】 | Text generation, summarization, translation tasks |
| **Prompt-tuning** | ~0.1% (soft prompts) 【19†L58-L61】 | Very low (like prefix) | Highly depends on model scale (closes gap at billions) 【19†L58-L61】 | NLU/NLG with huge LMs, multi-task zero-shot (FLAN, T0) |
| **Instruction Tuning** | ≈100% (usually full FT) | High (multi-task training over many datasets) | Improves zero/ few-shot generality; needs diverse tasks | Building generalist models (InstructGPT, FLAN, T0, etc.) |
| **Continual FT** | Varies (e.g. EWC: 100% + reg.) | High (if full FT), or low (if adapters per task) | Aims to retain past tasks; often requires replay data | Lifelong learning; updating models over time |
| **Multi-task FT** | 100% (shared + some heads) | High (training on many tasks simultaneously) | Efficient reuse of data; risk of task interference | Universal models for many tasks (mBERT, T5-type) |
| **Meta-learning** | Varies (often 100%) | Very high (nested optimizations) | Aims for extremely data-efficient adaptation | Research prototypes for few-shot; not common in practice |

*Notes:* The table entries are approximate. For example, Adapter modules can be added per layer and still only add a few percent of total weights【13†L50-L59】. LoRA's fraction depends on rank and layer count; in GPT-3 it was only ~37M params out of 175B【10†L25-L33】. Compute cost generally scales with how many weights are updated: full FT is worst, prefix/prompt lowest. Sample efficiency is often higher with PEFT methods (they overfit less and can train on fewer examples). The last column hints at common tasks: e.g., LoRA is popular for updating very large chat models, while prompt tuning is mostly used in research on frozen LMs.

## Open Questions and Evaluation Benchmarks

Despite progress, many research questions remain open. For example, **how to predict which fine-tuning method will work best** for a new task or dataset? The interplay between model size, data size, and adaptation method is not fully understood. Theoretical understanding of *transferability* – how much and which part of a model must change – is limited, though work on intrinsic dimension【43†L59-L67】 and delta-space【10†L49-L57】 offers clues. Another challenge is **multi-task adaptation**: current approaches often train on one task at a time, but ideally we want a model that seamlessly handles many (perhaps by scheduling or meta-learning) without forgetting【25†L1371-L1378】. Generating robust benchmarks for adaptation is also crucial: many tasks emphasize either classification or specific domains, but we lack standardized tests for, say, *continual learning* performance or *adversarial domain shift*.

To validate novel adaptation methods, we recommend the following benchmarks:

- **General NLU:** GLUE/SuperGLUE (text classification, inference), MMLU (massive multitask exam questions) – measure few-shot and fine-tune performance.
- **QA:** SQuAD v2, NaturalQuestions (answer extraction), TriviaQA.
- **Summarization/Generation:** CNN/DailyMail or XSum (metrics: ROUGE), GEM benchmark (human-evaluated generation).
- **Dialogue/Chat:** DSTC or PersonaChat (dialogue quality, coherence).
- **Domain Adaptation:** Domain-specific tasks like medical QA (e.g. PubMedQA), legal or finance sentiment sets. *Continual learning:* sequential task suites (e.g. permuted GLUE tasks).
- **Parameter-Efficient Metrics:** measure *update FLOPs per performance gain*, or *inference latency with adapters*. The PEFT literature often reports "trainable parameter fraction" and "GPU-hours to converge" on standard NLP tasks (e.g. GLUE tasks, language modeling on WikiText).

The above benchmarks cover English tasks; for other languages or modalities (code, speech) similar task families exist. Critically, evaluating a new method should involve comparing to strong baselines (full FT, best PEFT, and possibly best-in-class multi-task models) on multiple tasks and data regimes. Measuring *sample efficiency* (performance as a function of fine-tuning data size) is also important for novelty.

**In summary,** fine-tuning LLMs is an active area. Key references include the original Transformer and BERT papers【33†L99-L107】【13†L50-L59】, early prefix/prompt studies【17†L59-L64】【19†L58-L61】, and recent PEFT surveys【10†L25-L34】【10†L79-L88】. This report has drawn on these and others to outline a comprehensive view. Future work will continue to refine adaptation recipes and probe the limits of LLM transfer learning.

**References:** (Citations are given inline. Some key sources: Vaswani et al. 2017【33†L99-L107】; Houlsby et al. 2019 (adapters)【13†L50-L59】; Hu et al. 2021 (LoRA)【15†L60-L67】; Li & Liang 2021 (prefix)【17†L59-

L64】; Lester et al. 2021 (prompt)【19†L58-L61】; Wei et al. 2022 (FLAN instruction tuning)【48†L52-L60】; Gururangan et al. 2020 (domain/task pretraining)【38†L23-L32】; Ding et al. 2023 (delta-tuning overview)【10†L25-L34】; Aghajanyan et al. 2021 (intrinsic dim)【43†L59-L67】; Lei et al. 2023 (conditional adapters)【52†L52-L61】; Dettmers et al. 2023 (QLoRA)【54†L53-L61】; Chai et al. 2025 (augmentation survey)【55†L165-L173】【25†L1339-L1345】.)

---